APLICACIONES INTERACTIVAS

Gestor de Gimnasio

Etapa 1 — Back-end

Documentación técnica

Java · Spring Boot · Spring Data JPA · MySQL · HTTP / JSON · Postman

Integrantes: Ignacio Ugarte, Mateo Galli

Docentes: Juan Ignacio López, Luciano Pueyo

Fecha de entrega: 24/09/ 2026

Índice

1. Introducción 3
1.1 Qué cumple el sistema 3
2. Tecnologías utilizadas 4
2.1 HTTP y JSON 4
3. Proceso de desarrollo 5
4. Arquitectura en capas 6
4.1 Estructura de carpetas 6
5. Modelo de datos 7
5.1 Diagrama de entidades 7
5.2 Relaciones 7
5.3 Entidades y tablas 7
5.4 Creación de la base de datos 9
6. Responsabilidades de las clases 10
7. Sintaxis de los elementos utilizados 11
7.1 Anotaciones de las entidades (JPA) 11
7.2 Anotaciones de Spring 11
7.3 Anotaciones de validación 12
7.4 Lombok 12
7.5 Repository 12
7.6 Service: Optional e inyección de dependencias 12
7.7 Controller y ResponseEntity 13
8. Lógica de negocio 14
9. Documentación de endpoints 15
9.1 CRUD de cada entidad 15
9.2 Endpoints adicionales 15
9.3 Verbos HTTP utilizados 15
9.4 Ejemplos de peticiones 15
10. Validación y manejo de errores 17
11. Configuración y ejecución 18
11.1 application.properties 18
11.2 Pasos para ejecutar 18
12. Pruebas con Postman 19
13. Decisiones de diseño 20
14. Conclusión 21

1. Introducción

El **Gestor de Gimnasio** es el back-end de un sistema para administrar un gimnasio: los **planes** de membresía, las **clases** grupales, los **socios**, las **reservas** que hacen los socios para asistir a una clase y los **pagos** de las cuotas.

En esta etapa se construyó un **servidor** que recibe solicitudes por HTTP, las procesa, aplica la lógica de negocio y consulta o modifica los datos guardados en una base de datos SQL. El front-end no forma parte de la entrega: el cliente que se usa para probar y demostrar el sistema es **Postman**.

Aunque el servidor corre en `localhost`, el sistema está pensado como **cliente-servidor**: el cliente nunca se conecta a la base de datos. Solo conoce las rutas de la API (endpoints) y el formato JSON. Así, en la próxima etapa cualquier front-end (web, móvil o de escritorio) podrá usar este mismo back-end sin modificarlo.

1.1 Qué cumple el sistema

Requisito de la consigna	Cómo se cumple
Arquitectura por capas	Paquetes model, repository, service, controller y exception (sección 4).
Persistencia en base SQL	MySQL (XAMPP) con Spring Data JPA / Hibernate y un script SQL (sección 5).
Modelado de entidades	5 entidades: Plan, Clase, Socio, Reserva y Pago.
Relaciones entre entidades	Cuatro relaciones N a 1 y una N a N (sección 5.2).
CRUD de cada entidad	Crear, listar, buscar por id, modificar y borrar en las 5 entidades.
Un endpoint de cada tipo	GET, POST, PUT, PATCH y DELETE (sección 9).
Lógica de negocio	Reglas en la capa service (sección 8).
Validación de datos	Anotaciones @NotBlank, @Email, @Positive, etc. + @Valid (sección 10).
Manejo de errores	GlobalExceptionHandler: respuestas 400, 404 y 409 (sección 10).
Ejecución en localhost	Servidor Tomcat en http://localhost:8080 (sección 11).
Documentación de endpoints	Sección 9, README.md y colección de Postman.

2. Tecnologías utilizadas

Tecnología	Para qué se usa
Java 25	Lenguaje de programación del back-end.
Spring Boot 4	Framework indicado por la cátedra. Levanta el servidor web (Tomcat), conecta la base de datos y crea los objetos automáticamente (inyección de dependencias).
Spring Web MVC	Recibe las peticiones HTTP y las dirige al método del controller que corresponde.
Spring Data JPA + Hibernate	Convierte objetos Java en filas de tablas y genera el SQL automáticamente (ORM).
MySQL (XAMPP)	Sistema gestor de base de datos donde se guarda la información (base gimnasio_db).
Jakarta Validation	Valida los datos que llegan en las peticiones mediante anotaciones.
Jackson	Convierte automáticamente objetos Java a JSON y JSON a objetos Java.
Lombok	Genera getters, setters y constructores para no escribirlos a mano.
Maven	Descarga las librerías y construye el proyecto (archivo pom.xml).
Postman	Cliente HTTP para probar y demostrar la API.

2.1 HTTP y JSON

Cada operación de la API se identifica con **método HTTP + ruta** (por ejemplo `GET /api/socios`). Los datos viajan en **JSON**, un formato de texto liviano que entiende cualquier lenguaje. Spring convierte los objetos a JSON y viceversa sin que haya que programarlo. Ejemplo de una clase devuelta por la API:

```
{
  "id": 1,
  "nombre": "Spinning",
  "instructor": "Laura Gómez",
  "dia": "Lunes",
  "horario": "19:00",
  "cupo": 20
}
```

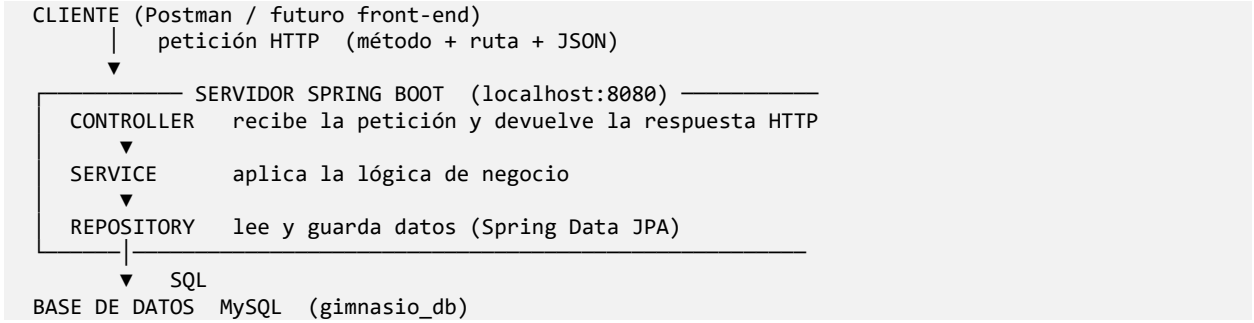
3. Proceso de desarrollo

El sistema se construyó de a una capa por vez, siguiendo el ejemplo visto en clase (Alumno – Materia – Carrera) y verificando cada paso antes de seguir.

1. **Proyecto base.** Se partió del proyecto Spring Boot generado para el TP, que ya tenía la entidad Socio con su CRUD y las clases vacías de las demás entidades.
2. **Modelado del dominio.** Se definieron las 5 entidades y cómo se relacionan: un socio tiene un plan, un plan incluye clases, un socio hace reservas a clases y registra pagos.
3. **Entidades (model).** Se programaron las clases con `@Entity`, sus atributos, las restricciones de columnas y las relaciones (`@ManyToOne`, `@ManyToMany`).
4. **Repositories.** Se creó una interfaz por entidad extendiendo `JpaRepository`, lo que da todas las operaciones básicas contra la base sin escribir SQL.
5. **Services.** Se programó la lógica de negocio de cada entidad: crear, buscar, modificar, borrar y las reglas propias del gimnasio.
6. **Controllers.** Se expusieron los endpoints REST con `@RestController`, usando los cinco verbos HTTP y devolviendo el código de estado adecuado.
7. **Validación y errores.** Se agregaron validaciones a las entidades y un manejador global de errores (`@RestControllerAdvice`).
8. **Base de datos.** Se configuró la conexión a MySQL (XAMPP) y se escribió el script `gimnasio_db.sql`, que crea las tablas y carga datos de ejemplo.
9. **Pruebas.** Se armó una colección de Postman con 35 peticiones que prueba todo el sistema de principio a fin. Todas dan el resultado esperado.

4. Arquitectura en capas

El sistema está dividido en **capas**, y cada una tiene una única responsabilidad. Una petición entra por el controller, pasa por el service y llega al repository, que habla con la base de datos. La respuesta hace el camino inverso.



Capa (paquete)	Responsabilidad	Qué NO hace
controller	Es la puerta de entrada. Define las rutas, recibe los datos de la petición, activa la validación y devuelve la respuesta con su código HTTP.	No tiene reglas de negocio ni habla con la base.
service	Contiene la lógica de negocio: verifica que existan los datos relacionados, aplica las reglas del gimnasio y decide qué guardar.	No sabe nada de HTTP.
repository	Acceso a datos. Spring genera automáticamente las consultas SQL.	No tiene reglas de negocio.
model	Entidades: cada clase representa una tabla y cada atributo una columna.	No tiene lógica.
exception	Manejo centralizado de errores: convierte las excepciones en respuestas HTTP claras.	—

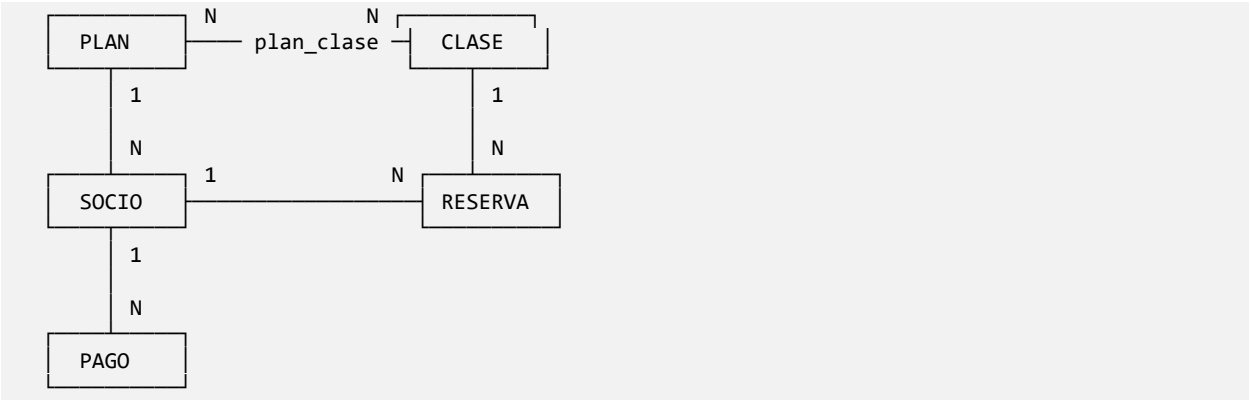
Regla de dependencias: cada capa solo usa a la de abajo. Controller → Service → Repository. El controller nunca llama directamente a un repository.

4.1 Estructura de carpetas

```
gestor_gimnasio/
├── pom.xml                        dependencias (Maven)
├── README.md                      guía rápida y lista de endpoints
├── database/gimnasio_db.sql       script de la base de datos
├── postman/gestor_gimnasio.postman_collection.json
└── src/main/
    ├── java/com/example/gestor_gimnasio/
    │   ├── GestorGimnasioApplication.java   arranca el servidor
    │   ├── model/         Plan, Clase, Socio, Reserva, Pago
    │   ├── repository/    PlanRepository, ClaseRepository, ...
    │   ├── service/       PlanService, ClaseService, ...
    │   ├── controller/    PlanController, ClaseController, ...
    │   └── exception/     GlobalExceptionHandler
    └── resources/application.properties     configuración
```

5. Modelo de datos

5.1 Diagrama de entidades



5.2 Relaciones

Relación	Tipo	Se lee	Cómo se implementa
Socio → Plan	N a 1	Muchos socios pueden tener el mismo plan.	@ManyToOne en Socio. Columna plan_id en la tabla socios.
Reserva → Socio	N a 1	Un socio puede hacer muchas reservas.	@ManyToOne en Reserva. Columna socio_id.
Reserva → Clase	N a 1	Una clase puede recibir muchas reservas.	@ManyToOne en Reserva. Columna clase_id.
Pago → Socio	N a 1	Un socio puede hacer muchos pagos.	@ManyToOne en Pago. Columna socio_id.
Plan ↔ Clase	N a N	Un plan incluye varias clases y una clase puede estar en varios planes.	@ManyToMany + @JoinTable en Plan. Tabla intermedia plan_clase.

Las relaciones son **unidireccionales**: solo la entidad que tiene la clave foránea conoce a la otra (Socio conoce su Plan, pero Plan no guarda la lista de socios). Esto simplifica el código y evita que el JSON entre en un ciclo infinito (plan → socios → plan → …).

Reserva funciona como el nexo entre Socio y Clase: además de unirlos, guarda un dato propio (la fecha a la que asiste el socio). Por eso es una entidad con dos `@ManyToOne` y no un `@ManyToMany`.

5.3 Entidades y tablas

Plan — tabla planes

Plan de membresía que contrata un socio.

Atributo	Tipo Java	Columna / restricción	Validación
id	Long	id, clave primaria, AUTO_INCREMENT	—
nombre	String	VARCHAR(100) NOT NULL	@NotBlank
precio	Double	DOUBLE NOT NULL	@NotNull, @Positive
clases	List<Clase>	Tabla intermedia plan_clase	Se maneja con endpoints propios

Clase — tabla clases

Clase grupal del gimnasio, con día, horario y cupo máximo.

Atributo	Tipo Java	Columna / restricción	Validación
id	Long	clave primaria	—
nombre	String	VARCHAR(100) NOT NULL	@NotBlank
instructor	String	VARCHAR(100) NOT NULL	@NotBlank
dia	String	VARCHAR(20) NOT NULL	@NotBlank (ej.: "Lunes")
horario	String	VARCHAR(5) NOT NULL	@NotBlank (ej.: "19:00")
cupo	Integer	INT NOT NULL	@NotNull, @Positive

Socio — tabla socios

Persona asociada al gimnasio. Es la entidad central del sistema.

Atributo	Tipo Java	Columna / restricción	Validación
id	Long	clave primaria	—
dni	String	VARCHAR(20) NOT NULL UNIQUE	@NotBlank
nombre / apellido	String	VARCHAR(100) NOT NULL	@NotBlank
email	String	VARCHAR(150) NOT NULL UNIQUE	@NotBlank, @Email
telefono	String	VARCHAR(30)	Opcional
fechaNacimiento	LocalDate	fecha_nacimiento DATE	Opcional
fechaIngreso	LocalDate	fecha_ingreso DATE	Si no se envía: hoy
plan	Plan	plan_id → FK a planes	Opcional
activo	Boolean	BIT NOT NULL	Al crear: true

Reserva — tabla reservas

Lugar que reserva un socio en una clase para una fecha.

Atributo	Tipo Java	Columna / restricción	Validación
id	Long	clave primaria	—
socio	Socio	socio_id → FK a socios	@NotNull
clase	Clase	clase_id → FK a clases	@NotNull
fecha	LocalDate	DATE NOT NULL	@NotNull

Pago — tabla pagos

Pago de una cuota realizado por un socio.

Atributo	Tipo Java	Columna / restricción	Validación
id	Long	clave primaria	—
socio	Socio	socio_id → FK a socios	@NotNull
monto	Double	DOUBLE NOT NULL	@NotNull, @Positive
fecha	LocalDate	DATE NOT NULL	Si no se envía: hoy
metodoPago	String	metodo_pago VARCHAR(30) NOT NULL	@NotBlank

5.4 Creación de la base de datos

La base se puede crear de dos formas que llegan al mismo resultado:

- **Script SQL** (`database/gimnasio_db.sql`): crea la base `gimnasio_db`, las 6 tablas con sus claves primarias, claves foráneas y restricciones UNIQUE, y carga datos de ejemplo (3 planes, 4 clases, 5 socios, 5 reservas y 5 pagos).
- **Automática con Hibernate**: con `spring.jpa.hibernate.ddl-auto=update`, al arrancar el servidor Hibernate lee las entidades y crea las tablas que falten.

Fragmento del script, que muestra la tabla intermedia de la relación N a N:

```
CREATE TABLE plan_clase (
    plan_id   BIGINT NOT NULL,
    clase_id  BIGINT NOT NULL,
    PRIMARY KEY (plan_id, clase_id),
    FOREIGN KEY (plan_id)  REFERENCES planes (id),
    FOREIGN KEY (clase_id) REFERENCES clases (id)
);
```

6. Responsabilidades de las clases

Clase	Capa	Responsabilidad
GestorGimnasioApplication	—	Tiene el método main. Arranca Spring y el servidor Tomcat en el puerto 8080.
Plan, Clase, Socio, Reserva, Pago	model	Representan las tablas de la base y sus relaciones. Contienen las validaciones de sus atributos.
PlanRepository, ClaseRepository, PagoRepository	repository	Acceso a datos con los métodos heredados de JpaRepository.
SocioRepository	repository	Además agrega findByActivo(Boolean) para filtrar socios activos o inactivos.
ReservaRepository	repository	Además agrega countByClaseIdAndFecha(...) para contar reservas y controlar el cupo.
PlanService	service	CRUD de planes y manejo de la relación N a N: agregar o quitar clases de un plan.
ClaseService	service	CRUD de clases.
SocioService	service	CRUD de socios. Activa al socio al crearlo, completa la fecha de ingreso, busca el plan asociado y cambia el estado (alta/baja lógica).
ReservaService	service	CRUD de reservas. Busca el socio y la clase y controla que el socio esté activo y que haya cupo.
PagoService	service	CRUD de pagos. Busca el socio y completa la fecha si no se envía.
PlanController … PagoController	controller	Uno por entidad. Exponen los endpoints REST (GET, POST, PUT, DELETE) y devuelven los códigos HTTP.
SocioController	controller	Además tiene PATCH /estado y GET /filtro.
PlanController	controller	Además tiene los endpoints de la relación N a N (POST y DELETE /planes/{id}/clases/{id}).
GlobalExceptionHandler	exception	Atrapa los errores de todos los controllers y responde 400, 404 o 409 con un mensaje claro.

7. Sintaxis de los elementos utilizados

7.1 Anotaciones de las entidades (JPA)

Anotación	Qué hace
@Entity	Indica que la clase se guarda en la base de datos.
@Table(name = "socios")	Nombre de la tabla.
@Id	Marca la clave primaria.
@GeneratedValue(strategy = IDENTITY)	El id lo genera la base automáticamente (AUTO_INCREMENT).
@Column(nullable = false, unique = true, length = 20)	Configura la columna: obligatoria (NOT NULL), sin repetidos (UNIQUE) y largo máximo.
@ManyToOne	Relación N a 1. Crea una clave foránea hacia la otra tabla.
@JoinColumn(name = "plan_id")	Nombre de la columna de la clave foránea.
@ManyToMany	Relación N a N.
@JoinTable(name = "plan_clase", ...)	Define la tabla intermedia de la relación N a N y sus dos columnas.

Ejemplo: la relación N a 1 de Socio con Plan.

```
@ManyToOne
@JoinColumn(name = "plan_id")
private Plan plan;
```

Ejemplo: la relación N a N de Plan con Clase.

```
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(
    name = "plan_clase",
    joinColumns = @JoinColumn(name = "plan_id"),
    inverseJoinColumns = @JoinColumn(name = "clase_id")
)
private List<Clase> clases = new ArrayList<>();
```

`fetch = FetchType.EAGER` hace que al leer un plan se traigan también sus clases, para poder mostrarlas en el JSON.

7.2 Anotaciones de Spring

Anotación	Qué hace
@SpringBootApplication	Arranca la aplicación y busca automáticamente todas las clases del proyecto.
@RestController	La clase atiende peticiones HTTP y lo que devuelve se convierte en JSON.
@RequestMapping("/api/socios")	Ruta base de todos los endpoints del controller.
@GetMapping, @PostMapping, @PutMapping, @PatchMapping, @DeleteMapping	Asocian un método a un verbo HTTP y una ruta.
@PathVariable	Toma un valor de la ruta. En /api/socios/5, id vale 5.
@RequestParam	Toma un parámetro de la URL. En /api/socios/filtro?activo=true, activo vale true.
@RequestBody	Convierte el JSON que envía el cliente en un objeto Java.
@Valid	Ejecuta las validaciones de la entidad antes de entrar al método.
@Service	Marca la clase como servicio, para que Spring la cree y la inyecte.

Anotación	Qué hace
@Transactional	Todo el método se ejecuta como una sola operación: si algo falla, no se guarda nada a medias.
@RestControllerAdvice / @ExceptionHandler	Clase que atrapa errores de todos los controllers / método que maneja un tipo de error.

7.3 Anotaciones de validación

Anotación	Regla
@NotBlank	El texto es obligatorio y no puede estar vacío.
@NotNull	El valor es obligatorio.
@Email	Debe tener formato de email.
@Positive	El número debe ser mayor a cero.

7.4 Lombok

`@Data` genera en la compilación los getters, setters, `toString`, `equals` y `hashCode` de todos los atributos. Así las entidades quedan cortas y fáciles de leer.

7.5 Repository

Un repository es una **interfaz vacía** que extiende `JpaRepository<Entidad, TipoDelId>`. Spring crea la implementación y el SQL automáticamente:

```
public interface SocioRepository extends JpaRepository<Socio, Long> {

    // Spring arma la consulta a partir del nombre del método:
    // SELECT * FROM socios WHERE activo = ?
    List<Socio> findByActivo(Boolean activo);
}
```

Método heredado	SQL que genera
findAll()	SELECT * FROM socios
findById(id)	SELECT * FROM socios WHERE id = ?
save(socio)	INSERT si el socio no tiene id; UPDATE si ya lo tiene
deleteById(id)	DELETE FROM socios WHERE id = ?
existsById(id)	Consulta si existe un registro con ese id

7.6 Service: Optional e inyección de dependencias

```
@Service
@Transactional
public class SocioService {

    private final SocioRepository socioRepository;

    // Inyección de dependencias: Spring crea el repository y lo pasa por el constructor
    public SocioService(SocioRepository socioRepository) {
        this.socioRepository = socioRepository;
    }

    public Optional<Socio> findById(Long id) {
        return socioRepository.findById(id);
    }
}
```

Inyección de dependencias: el service no hace `new SocioRepository()`. Spring crea el objeto y lo entrega por el constructor.

Optional: es una "caja" que puede contener un socio o estar vacía. Obliga al controller a contemplar el caso en que el id no existe y a responder 404 en vez de fallar.

7.7 Controller y ResponseEntity

```
@GetMapping("/{id}")                                    // GET /api/socios/5
public ResponseEntity<Socio> getById(@PathVariable Long id) {
    Optional<Socio> encontrado = socioService.findById(id);

    if (encontrado.isPresent()) {
        return ResponseEntity.ok(encontrado.get());    // 200 + socio en JSON
    }
    return ResponseEntity.notFound().build();          // 404 sin cuerpo
}

@PostMapping                                            // POST /api/socios
public ResponseEntity<Socio> create(@Valid @RequestBody Socio socio) {
    Socio creado = socioService.save(socio);
    return ResponseEntity.status(HttpStatus.CREATED).body(creado);   // 201
}
```

`ResponseEntity` permite elegir el **código de estado HTTP** de la respuesta además de su contenido.

8. Lógica de negocio

Las reglas del gimnasio están en la capa **service**:

Entidad	Regla	Si no se cumple
Socio	Todo socio nuevo empieza activo.	—
Socio	Si no se informa la fecha de ingreso, se usa la fecha de hoy.	—
Socio	El plan informado tiene que existir.	404
Socio	Baja y alta lógica con PATCH: el socio no se borra, se marca como inactivo.	—
Socio	DNI y email no se pueden repetir (UNIQUE en la base).	409
Reserva	El socio y la clase tienen que existir.	404
Reserva	Un socio inactivo no puede reservar.	409
Reserva	Una clase no acepta más reservas que su cupo para una misma fecha.	409
Pago	El socio tiene que existir. Si no se informa la fecha, se usa la de hoy.	404
Plan	Solo se agregan al plan clases que existen, y no se repiten.	404
Plan / Clase	No se puede borrar un plan con socios, ni una clase con reservas o incluida en un plan (lo impiden las claves foráneas).	409

Ejemplo del control de cupo en `ReservaService`:

```
// 1. Un socio dado de baja no puede reservar
if (!socio.getActivo()) {
    throw new IllegalStateException("El socio está inactivo y no puede reservar");
}

// 2. La clase no puede superar su cupo en esa fecha
long reservasDelDia = reservaRepository
        .countByClaseIdAndFecha(clase.getId(), reserva.getFecha());
if (reservasDelDia >= clase.getCupo()) {
    throw new IllegalStateException("La clase no tiene cupo disponible para esa fecha");
}
```

9. Documentación de endpoints

URL base: `http://localhost:8080`. Las peticiones con cuerpo se envían en JSON (en Postman: Body → raw → JSON).

9.1 CRUD de cada entidad

Las cinco entidades tienen los mismos cinco endpoints. `{entidad}` puede ser `planes`, `clases`, `socios`, `reservas` o `pagos`.

Método	Ruta	Operación	Respuestas
GET	/api/{entidad}	Listar todos (Read)	200
GET	/api/{entidad}/{id}	Buscar uno por id (Read)	200, 404
POST	/api/{entidad}	Crear (Create)	201, 400, 404, 409
PUT	/api/{entidad}/{id}	Modificar todos los datos (Update)	200, 400, 404
DELETE	/api/{entidad}/{id}	Borrar (Delete)	204, 404, 409

9.2 Endpoints adicionales

Método	Ruta	Qué hace	Respuestas
PATCH	/api/socios/{id}/estado?activo=false	Modificación parcial: da de baja o de alta al socio.	200, 404
GET	/api/socios/filtro?activo=true	Lista solo socios activos (o inactivos).	200
POST	/api/planes/{planId}/clases/{claseId}	Agrega una clase al plan (N a N).	200, 404
DELETE	/api/planes/{planId}/clases/{claseId}	Quita una clase del plan (N a N).	200, 404

9.3 Verbos HTTP utilizados

Verbo	Significado	Ejemplo
GET	Leer datos	GET /api/socios
POST	Crear un recurso	POST /api/clases
PUT	Reemplazar todos los datos de un recurso	PUT /api/planes/1
PATCH	Modificar solo una parte	PATCH /api/socios/1/estado?activo=false
DELETE	Borrar un recurso	DELETE /api/pagos/3

9.4 Ejemplos de peticiones

Las relaciones se envían indicando solo el **id** del objeto relacionado. El service busca el objeto completo en la base.

```
POST /api/planes
{ "nombre": "Plan Full", "precio": 35000 }

POST /api/clases
{ "nombre": "Spinning", "instructor": "Laura Gómez",
  "dia": "Lunes", "horario": "19:00", "cupo": 20 }

POST /api/socios
{ "dni": "40123456", "nombre": "Juan", "apellido": "Pérez",
  "email": "juan@mail.com", "plan": { "id": 1 } }
```

```
POST /api/reservas
{ "socio": { "id": 1 }, "clase": { "id": 1 }, "fecha": "2026-10-05" }

POST /api/pagos
{ "socio": { "id": 1 }, "monto": 35000, "metodoPago": "Transferencia" }
```

Respuesta de `GET /api/socios/1` (resumida). El socio trae su plan (N a 1) y el plan trae sus clases (N a N):

```
{
  "id": 1,
  "nombre": "Lucía",
  "apellido": "Fernández",
  "activo": true,
  "plan": {
    "id": 2,
    "nombre": "Plan Full",
    "precio": 35000.0,
    "clases": [
      { "id": 1, "nombre": "Spinning", "dia": "Lunes", "horario": "19:00", "cupo": 20 },
      { "id": 2, "nombre": "Yoga",     "dia": "Martes", "horario": "09:30", "cupo": 15 }
    ]
  }
}
```

10. Validación y manejo de errores

Cuando el controller recibe un objeto con `@Valid`, Spring revisa las anotaciones de la entidad. Si algún dato no cumple, no se ejecuta el método y se responde **400**. Todos los errores pasan por `GlobalExceptionHandler`, que devuelve un JSON claro en lugar de un error interno.

Código	Cuándo ocurre	Origen
200 OK	Consulta o modificación correcta.	ResponseEntity.ok(...)
201 Created	Se creó un registro.	ResponseEntity.status(CREATED)
204 No Content	Se borró un registro.	ResponseEntity.noContent()
400 Bad Request	Faltan datos o son inválidos.	MethodArgumentNotValidException
404 Not Found	El id buscado (o el de una relación) no existe.	Optional vacío / NoSuchElementException
409 Conflict	Se rompe una regla de negocio (socio inactivo, sin cupo) o la base rechaza la operación (DNI repetido, borrar un plan con socios).	IllegalStateException / DataIntegrityViolationException

Ejemplo de respuesta 400 al crear un socio sin nombre y con email inválido:

```
{
  "nombre": "El nombre es obligatorio",
  "email": "El email no es válido"
}
```

Ejemplo de respuesta 409:

```
{ "error": "El socio está inactivo y no puede reservar" }
```

11. Configuración y ejecución

11.1 application.properties

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/gimnasio_db
    ?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Propiedad	Explicación
server.port	Puerto donde escucha el servidor.
spring.datasource.url	Dirección de la base. createDatabaseIfNotExist=true la crea si no existe.
username / password	Usuario de MySQL. En XAMPP es root, sin contraseña.
ddl-auto=update	Hibernate crea o actualiza las tablas a partir de las entidades.
show-sql=true	Muestra en la consola el SQL que ejecuta Hibernate.

11.2 Pasos para ejecutar

1. Abrir XAMPP y hacer clic en **Start** en MySQL.
2. (Opcional) Importar `database/gimnasio_db.sql` para cargar los datos de ejemplo.
3. Ejecutar `GestorGimnasioApplication` desde IntelliJ.
4. Esperar el mensaje "Started GestorGimnasioApplication" en la consola.
5. Probar `http://localhost:8080/api/socios` en el navegador o usar Postman.

12. Pruebas con Postman

Se entrega la colección `postman/gestor_gimnasio.postman_collection.json` con **35 peticiones** organizadas en carpetas, que se ejecutan en orden con "Run collection". Todas dan el resultado esperado.

Carpeta	Qué prueba
1. Planes	POST, GET todos, GET por id, PUT.
2. Clases	POST, GET todos, GET por id, PUT.
3. Relación N a N	Agregar una clase a un plan y ver el plan con sus clases.
4. Socios	CRUD, socio con plan (N a 1) y filtro por estado.
5. Reservas	CRUD, reserva con socio y clase (N a 1).
6. Pagos	CRUD, pago con socio (N a 1).
7. PATCH y errores	Baja lógica con PATCH, socio inactivo que no puede reservar (409), validación (400), id inexistente (404), borrar un plan con socios (409).
8. DELETE	Borra todo lo creado por la colección.

Cada POST guarda automáticamente el id creado en una variable (`{{planId}}`, `{{socioId}}`, …) que usan las peticiones siguientes, y cada petición comprueba el código HTTP esperado.

13. Decisiones de diseño

Decisión	Justificación
Arquitectura en capas	Cada capa tiene una sola responsabilidad. El código es más fácil de entender, probar y modificar, y el mismo back-end puede usarlo cualquier front-end.
Seguir el modelo visto en clase	Plan–Clase replica Alumno–Materias (N a N) y Socio–Plan replica Materia–Carrera (N a 1). Se usan las mismas herramientas y estilo que el ejemplo de la cátedra.
Exponer las entidades directamente (sin DTOs)	Mantiene el sistema simple, con menos clases. Las relaciones unidireccionales evitan los ciclos infinitos en el JSON.
Reserva como entidad	La relación entre Socio y Clase necesita un dato propio (la fecha), que no se puede guardar en una tabla @ManyToMany.
Relaciones enviadas por id	El cliente solo manda { "id": 1 } y el service busca el objeto completo. Así se garantiza que el dato relacionado exista.
PATCH para el estado del socio	Cambiar solo el estado es una modificación parcial, y PATCH es el verbo correcto para eso. Además permite una baja lógica sin perder el historial.
Optional + ResponseEntity	Permiten responder 404 cuando algo no existe en vez de devolver null o un error 500.
Manejo de errores centralizado	Una sola clase traduce los errores a códigos HTTP. Los controllers no se llenan de try/catch.
@Transactional en los services	Si una operación falla a mitad de camino, la base vuelve al estado anterior.
Inyección por constructor	Las dependencias quedan explícitas y Spring se encarga de crearlas.
XAMPP como gestor de base	Gratuito, fácil de instalar y trae MySQL listo para usar.

14. Conclusión

Se desarrolló un back-end REST completo y separado del cliente: cinco entidades con relaciones N a 1 y N a N, CRUD completo para cada una, los cinco verbos HTTP, validación de datos, reglas de negocio y manejo de errores. Los datos se guardan en MySQL y todo el funcionamiento se puede demostrar en vivo con la colección de Postman. El sistema queda listo para que en la próxima etapa se le conecte un front-end sin cambiar el servidor.